

AIAC

Lab Assignment-8

Name : Anoop
Hallticket:2303A51085
Batch-02

1. (Username Validator – Apply AI in Authentication Context)

Task:

Use AI to generate at least 3 assert test cases for a function `is_valid_username(username)` and then implement the function using Test-Driven Development principles.

Requirements:

- Username length must be between 5 and 15 characters.
- Must contain only alphabets and digits.
- Must not start with a digit.
- No spaces allowed.

Example Assert Test

Cases:

- `assert is_valid_username("User123") == True`
- `assert is_valid_username("12User") == False`
- `assert is_valid_username("Us er") == False`

Expected Output:

- Username validation logic successfully passing all AI-generated test cases.

```
week 8 > valid_username.py > ...
1  def is_valid_username(username):
2      if len(username) < 5 or len(username) > 16:
3          return False
4      if not username[0].isalpha():
5          return False
6      for char in username:
7          if not (char.isalnum() or char == '_'):
8              return False
9      return True
10 # Test cases
11 assert is_valid_username("User123") == True
12 assert is_valid_username("12User") == False
13 assert is_valid_username("Us er") == False
14 print("Username validation logic successfully passing all AI-generated test cases")
```

Output:

```
PS C:\Users\User\OneDrive\Desktop\AIAC> & C:/Users/User/AppData/Local/Python/python
IAC/week 8/valid_username.py"
Username validation logic successfully passing all AI-generated test cases
PS C:\Users\User\OneDrive\Desktop\AIAC>
```

2. (Even–Odd & Type Classification – Apply AI for Robust Input Handling) Task:

Use AI to generate at least 3 assert test cases for a function `classify_value(x)` and implement it using conditional logic and loops.

Requirements:

- If input is an integer, classify as "Even" or "Odd".
- If input is 0, return "Zero".
- If input is non-numeric, return "Invalid Input".

Example Assert Test Cases:

- `assert classify_value(8) == "Even"`
- `assert classify_value("abc") == "Invalid Input"`
- Expected Output
- `assert classify_value("abc") == "Invalid Input"`

Expected Output:

- Function correctly classifying values and passing all test cases

week 8 >  classify_value.py > ...

```
1 def classify_value(x):
2     if isinstance(x, int):
3         if x == 0:
4             return "Zero"
5         elif x % 2 == 0:
6             return "Even"
7         else:
8             return "Odd"
9     else:
10        return "Invalid Input"
11 # Test cases
12 assert classify_value(0) == "Zero"
13 assert classify_value(8) == "Even"
14 assert classify_value(7) == "Odd"
15 assert classify_value("abc") == "Invalid Input"
16 print("Function correctly classifying values and passing all test cases.")
```

Output

```
PS C:\Users\User\OneDrive\Desktop\AIAC> & C:/Users/User/AppData/Local/Programs/Microsoft AIAC/week 8/classify_value.py
Function correctly classifying values and passing all test cases.
PS C:\Users\User\OneDrive\Desktop\AIAC>
```

3.(Palindrome Checker – Apply AI for String Normalization)

Task:

Use AI to generate at least 3 assert test cases for a function `is_palindrome(text)` and implement the function.

Requirements:

- Ignore case, spaces, and punctuation.
- Handle edge cases such as empty strings and single characters.

Example Assert Test Cases:

- `assert is_palindrome("Madam") == True`
- `assert is_palindrome("A man a plan a canal Panama") == True`
- `assert is_palindrome("Python") == False`
- Expected Output #3:
- Function correctly identifying palindromes and passing all AI-generated tests.

```
week 8 > is_palindrome.py > ...
1  def is_palindrome(text):
2      # Remove spaces and punctuation, and convert to lowercase
3      cleaned_text = ''.join(char.lower() for char in text if char.isalnum())
4
5      # Check if the cleaned text is equal to its reverse
6      return cleaned_text == cleaned_text[::-1]
7
8  # Test cases
9  assert is_palindrome("Madam") == True
10 assert is_palindrome("A man a plan a canal Panama") == True
11 assert is_palindrome("Python") == False
12 print("Function correctly identifying palindromes and passing all AI-generated tests")
```

Output:

```
PS C:\Users\User\OneDrive\Desktop\AIAC> & C:/Users/User/AppData/Local/Python/
IAC/week 8/is_palindrome.py
Function correctly identifying palindromes and passing all AI-generated tests
PS C:\Users\User\OneDrive\Desktop\AIAC>
```

2. (Email ID Validation – Apply AI for Data

Validation) Task:

Use AI to generate at least 3 assert test cases for a function `validate_email(email)` and implement the function.

Requirements:

- Must contain @ and .
- Must not start or end with special characters.
- Should handle invalid formats gracefully.

Example Assert Test Cases:

- `assert validate_email("user@example.com") == True`
- `assert validate_email("userexample.com") == False`
- `assert validate_email("@gmail.com") == False`
- Expected Output #4:

- Email validation function passing all AI-generated test cases and handling edge cases correctly.

```

week 8 > validate_email.py > validate_email
1 def validate_email(email):
2     if '@' in email and '.' in email:
3         if email[0].isalnum() and email[-1].isalnum():
4             return True
5     return False
6 # Test cases
7 assert validate_email("user@example.com") == True
8 assert validate_email("userexample.com") == False
9 assert validate_email("@gmail.com") == False
10 print("Email validation function passing all AI-generated test cases and handling edge cases correctly.")

```

Output:

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE
PS C:\Users\User\OneDrive\Desktop\AIAC> & C:/Users/User/AppData/Local/Python/pythoncore-3.14-64/
IAC/week 8/validate_email.py"
Email validation function passing all AI-generated test cases and handling edge cases correctly.
PS C:\Users\User\OneDrive\Desktop\AIAC>

```

3. (Perfect Number Checker – Test Case

Design) Function:

Check if a number is a perfect number (sum of divisors = number).

Test Cases to Design:

- Normal case: 6 → True, 10 → False.
- Edge case: 1.
- Negative number case.
- Larger case: 28.

Requirement:

```

week 8 > perfect_num_checker.py > ...
1 def perfect_num_checker(n):
2     if n < 2:
3         return False
4     sum_of_divisors = 1
5     for i in range(2, n):
6         if n % i == 0:
7             sum_of_divisors += i
8     return sum_of_divisors == n
9 #test cases
10 #normal cases
11 assert perfect_num_checker(6) == True
12 assert perfect_num_checker(10) == False
13 assert perfect_num_checker(28) == True
14 assert perfect_num_checker(-1) == False
15 print("All test cases passed!")

```

- Output:

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL

PS C:\Users\User\OneDrive\Desktop\AIAC> & C:/U
IAC/week 8/perfect_num_check.py"
All test cases passed!
PS C:\Users\User\OneDrive\Desktop\AIAC>

```

4. (Abundant Number Checker – Test Case

Design) Function:

Check if a number is abundant (sum of divisors > number).

Test Cases to Design:

- Normal case: 12 → True, 15 → False.
- Edge case: 1.
- Negative number case.
- Large case: 945.

Requirement:

Validate correctness with unittest

```

week 8 > abundant_num.py > ...
1  import unittest
2  def abundant_num(n):
3      if n < 1:
4          return False
5      sum_of_divisors = 0
6      for i in range(1, n):
7          if n % i == 0:
8              sum_of_divisors += i
9      return sum_of_divisors > n
10 class TestAbundantNum(unittest.TestCase):
11     def test_abundant_num(self):
12         self.assertTrue(abundant_num(12))
13         self.assertFalse(abundant_num(15))
14     def test_abundant_num_edge_cases(self):
15         self.assertFalse(abundant_num(1))
16     def test_abundant_num_negative(self):
17         self.assertFalse(abundant_num(-2))
18     def test_abundant_num_large(self):
19         self.assertFalse(abundant_num(946))
20 if __name__ == '__main__':
21     unittest.main()
22
PS C:\Users\User\OneDrive\Desktop\AIAC> & C:/U
IAC/week 8/abundant_num.py"
....
-----
Ran 4 tests in 0.003s

OK
PS C:\Users\User\OneDrive\Desktop\AIAC>

```

5.

6. (Deficient Number Checker – Test Case

Design) Function:

Check if a number is deficient (sum of divisors < number).

Test Cases to Design:

- Normal case: 8 → True, 12 → False.
- Edge case: 1.
- Negative number case.
- Large case: 546.

Requirement:

Validate correctness with pytest.

```
number_checker.py X
week 8 > number_checker.py > ...
1  #Check if a number is deficient (sum of divisors < number)
2  def number_checker(n):
3      if n < 1:
4          return False
5      sum_of_divisors = 0
6      for i in range(1, n):
7          if n % i == 0:
8              sum_of_divisors += i
9      return sum_of_divisors < n
10 #test cases
11 def test_num_check():
12     assert number_checker(8) == True
13     assert number_checker(12) == False
14     assert number_checker(1) == False
15     assert number_checker(-5) == False
16     assert number_checker(546) == False
17 print("All test cases passed!")
```

Output:

```
PS C:\Users\User\OneDrive\Desktop\AIAC\week 8> python -m pytest number_checker.py
===== test session starts =====
platform win32 -- Python 3.14.2, pytest-9.0.2, pluggy-1.6.0
rootdir: C:\Users\User\OneDrive\Desktop\AIAC\week 8
collected 1 item

number_checker.py .

===== 1 passed in 1.72s =====
PS C:\Users\User\OneDrive\Desktop\AIAC\week 8>
```

7. Write a function LeapYearChecker and validate its implementation using 10 pytest test cases.

```

week 8 > leap_yr.py > test_leap_year_checker
1  #Write a function LeapYearChecker and validate its implementation using 7
2  def leap_year_checker(year):
3      if (year % 4 == 0 and year % 100 != 0) or
4          return True
5      return False
6  # Test cases
7  def test_leap_year_checker():
8      assert leap_year_checker(2020) == True
9      assert leap_year_checker(1900) == False
10     assert leap_year_checker(2000) == True
11     assert leap_year_checker(2021) == False
12     assert leap_year_checker(2400) == True
13     assert leap_year_checker(1800) == False
14     assert leap_year_checker(1996) == True
15     assert leap_year_checker(2100) == False
16     assert leap_year_checker(1600) == True
17     assert leap_year_checker(2024) == True

```

Output:

```

PS C:\Users\User\OneDrive\Desktop\AIAC\week 8> python -m pytest leap_yr.py
===== test session starts =====
platform win32 -- Python 3.14.2, pytest-9.0.2, pluggy-1.6.0
rootdir: C:\Users\User\OneDrive\Desktop\AIAC\week 8
collected 1 item

leap_yr.py .

===== 1 passed in 0.50s =====

```

8. Write a function SumOfDigits and validate its implementation using 7 pytest test cases.

```

week 8 > sod.py > test_sum_of_digits
2  def sum_of_digits(n):
3      return sum(int(digit) for digit in str(abs(n)))
4  # Test cases
5  def test_sum_of_digits():
6      assert sum_of_digits(123) == 6
7      assert sum_of_digits(-456) == 15
8      assert sum_of_digits(0) == 0
9      assert sum_of_digits(789) == 24
10     assert sum_of_digits(-10) == 1
11     assert sum_of_digits(999) == 27
12     assert sum_of_digits(-12345) == 15
13     print("All test cases passed!")
14

```

Output:

```

=====
PS C:\Users\User\OneDrive\Desktop\AIAC\week 8> python -m pytest sod.py
===== test session starts =====
platform win32 -- Python 3.14.2, pytest-9.0.2, pluggy-1.6.0
rootdir: C:\Users\User\OneDrive\Desktop\AIAC\week 8
collected 1 item

sod.py . [100%]

===== 1 passed in 0.16s =====

```

9. Write a function SortNumbers (implement bubble sort) and validate its implementation using 25 pytest test cases.

```

week 8 > bubble_sort.py > test_bubble_sort
2 def bubble_sort(arr):
3     for j in range(0, n-i-1):
4         if arr[j] > arr[j+1]:
5             arr[j], arr[j+1] = arr[j+1], arr[j]
6     return arr
7
8 #test cases
9 def test_bubble_sort():
10     assert bubble_sort([64, 34, 25, 12, 22, 11, 90]) == [11, 12, 22, 25, 34, 64, 90]
11     assert bubble_sort([5, 1, 4, 2, 8]) == [1, 2, 4, 5, 8]
12     assert bubble_sort([]) == []
13     assert bubble_sort([1]) == [1]
14     assert bubble_sort([3, 2]) == [2, 3]
15     assert bubble_sort([10, -1, 0]) == [-1, 0, 10]
16     assert bubble_sort([5, -5, -10]) == [-10, -5, 5]
17     assert bubble_sort([1.5, 0.5, -1.0]) == [-1.0, 0.5, 1.5]
18     assert bubble_sort(['banana', 'apple', 'cherry']) == ['apple', 'banana', 'cherry']
19     assert bubble_sort(['z', 'a', 'm']) == ['a', 'm', 'z']
20     print("All test cases passed!")
21

```

Output:

```

===== 1 passed in 0.16s =====
PS C:\Users\User\OneDrive\Desktop\AIAC\week 8> python -m pytest bubble_sort.py
===== test session starts =====
platform win32 -- Python 3.14.2, pytest-9.0.2, pluggy-1.6.0
rootdir: C:\Users\User\OneDrive\Desktop\AIAC\week 8
collected 1 item

bubble_sort.py . [100%]

===== 1 passed in 0.18s =====

```

10. Write a function ReverseString and validate its implementation using 5 unittest test cases.


```

week 8 > rev_string.py > reverse_string
1  def reverse_string(s):
2      return s[::-1]
3  class TestReverseString(unittest.TestCase):
4      def test_reverse_string(self):
5          self.assertEqual(reverse_string("hello"), "olleh")
6          self.assertEqual(reverse_string("Python"), "nohtyP")
7          self.assertEqual(reverse_string(""), "")
8          self.assertEqual(reverse_string("a"), "a")
9          self.assertEqual(reverse_string("12345"), "54321")
10 if __name__ == '__main__':
11     unittest.main()
12
13

```

Output:

```

PS C:\Users\User\OneDrive\Desktop\AIAC\week 8> & C:/Users/User/AppData/Local/Python/pythoncore-3.14-64/python.exe "
:/Users/User/OneDrive/Desktop/AIAC/week 8/rev_string.py"
.
-----
Ran 1 test in 0.002s

OK
PS C:\Users\User\OneDrive\Desktop\AIAC\week 8>

```

11. Write a function AnagramChecker and validate its implementation using 10 unittest test cases.

```

week 8 > anagram_checker1.py > ...
1  import unittest
2  def anagram_checker(str1, str2):
3      return sorted(str1.replace(" ", "").lower()) == sorted(str2.replace(" ", "").lower())
4  class TestAnagramChecker(unittest.TestCase):
5      def test_anagram_checker(self):
6          self.assertTrue(anagram_checker("listen", "silent"))
7          self.assertTrue(anagram_checker("Triangle", "Integral"))
8          self.assertFalse(anagram_checker("Apple", "Pabble"))
9          self.assertTrue(anagram_checker("Dormitory", "Dirty Room"))
10         self.assertFalse(anagram_checker("Hello", "World"))
11         self.assertTrue(anagram_checker("Clint Eastwood", "Old Clint Eastwood"))
12         self.assertFalse(anagram_checker("Python", "Java"))
13         self.assertTrue(anagram_checker("Astronomer", "Moon Star"))
14         self.assertFalse(anagram_checker("Test", "Best"))
15         self.assertTrue(anagram_checker("Conversation", "Voice Conversation"))
16 if __name__ == '__main__':
17     unittest.main()

```

Output:

```
PS C:\Users\User\OneDrive\Desktop\AIAC\week 8> & C:/Users/User/AppData/Local/Python/pythoncore-3.14-64/python.exe "c:/Users/User/OneDrive/Desktop/AIAC/week 8/anagram_checker1.py"
.
-----
Ran 1 test in 0.001s

OK
PS C:\Users\User\OneDrive\Desktop\AIAC\week 8>
```

12. Write a function ArmstrongChecker and validate its implementation using 8 unittest test cases

```
ort.py | rev_string.py | anagram_checker1.py | armstrong.py X
week 8 > armstrong.py > arm_strong_checker
1 import unittest
2 def arm_strong_checker(n):
3     num_str = str(n)
4     num_digits = len(num_str)
5     armstrong_sum = sum(int(digit) ** num_digits for digit in num_str)
6     return armstrong_sum == n
7 class TestArmstrongChecker(unittest.TestCase):
8     def test_armstrong_checker(self):
9         self.assertTrue(arm_strong_checker(153))
10        self.assertTrue(arm_strong_checker(370))
11        self.assertTrue(arm_strong_checker(371))
12        self.assertTrue(arm_strong_checker(407))
13        self.assertFalse(arm_strong_checker(123))
14        self.assertFalse(arm_strong_checker(0))
15        self.assertFalse(arm_strong_checker(-153))
16        self.assertFalse(arm_strong_checker(9474)) # 9474 is not
17 if __name__ == '__main__':
18     unittest.main()
19
```

Output:

```
PS C:\Users\User\OneDrive\Desktop\AIAC> & C:/Users/User/AppData/Local/Python/pythoncore-3.14-64/python.exe "c:/Users/User/OneDrive/Desktop/AIAC/week 8/armstrong.py"
.
-----
Ran 1 test in 0.002s

OK
PS C:\Users\User\OneDrive\Desktop\AIAC>
```